

NumPy & Pandas 자기주도 학습 가이드

Part A. NumPy

1. NumPy란 무엇인가
2. ndarray와 4가지 속성
3. 배열을 만드는 함수들
4. 인덱싱과 슬라이싱 - 그리고 View라는 함정
5. Boolean Indexing과 Fancy Indexing
6. `reshape` - 모양 바꾸기
7. 배열 연산과 브로드캐스팅
8. 유니버설 함수 (`ufunc`) - 가볍게 훑기
9. 집계 함수와 `axis`

Part B. Pandas

10. Pandas란 - `Series`와 `DataFrame`
11. `DataFrame` 만들기
12. Inspecting - 데이터와의 첫인사
13. 열 선택과 행 선택 - `[]`, `loc`, `iloc`
14. 조건 필터링
15. 전처리 - 결측치와 타입 다루기
16. `groupby`와 `pivot_table` - 표를 요약

정리

NumPy & Pandas 자기주도 학습 가이드

이 문서의 사용법

위에서부터 순서대로 읽으며, 함께 제공되는 `numpy_pandas_guide.py`를 실행해 출력 결과를 눈으로 확인하세요.
각 장의 번호는 파이썬 파일의 CHAPTER 번호와 1:1로 대응합니다.
다 읽은 뒤 실습 노트북(실습2)의 TODO를 풀면, 오늘 배운 내용이 그대로 문제로 나옵니다.

목표

"반복문 없이 데이터 전체를 한 번에 계산하고(NumPy), 엑셀처럼 표를 다룬다(Pandas)"

Part A. NumPy

1. NumPy란 무엇인가

NumPy(Numerical Python)는 파이썬에서 숫자 계산을 빠르게 하기 위한 라이브러리입니다.

Reason: 파이썬 리스트는 원소를 하나씩 반복하며 계산하지만, NumPy 배열은 같은 자료형의 숫자를 메모리에 연속으로 저장하고 내부가 C로 구현되어 있어, 배열 전체에 연산을 한 번에 적용합니다. 데이터가 커질수록 격차가 벌어져 수십 배 이상 빨라집니다.

Example: 리스트와 배열에 `* 2`를 했을 때의 차이가 모든 것을 말해 줍니다.

```
import numpy as np

py_list = [1, 2, 3]
print(py_list * 2)           # [1, 2, 3, 1, 2, 3] => 이어붙이기 (계산이 아님!)

np_arr = np.array([1, 2, 3])
print(np_arr * 2)           # [2 4 6]           => 원소별 2배 (진짜 계산)
```

Point(재확인): 리스트의 `*`는 "복사해서 이어붙여라", 배열의 `*`는 "각 원소를 곱해라". 이 차이가 NumPy를 쓰는 이유의 전부입니다.

비유 - 리스트는 "종이 서류 묶음"이라 한 장씩 넘기며 도장을 찍어야 하고, ndarray 는 "엑셀 열 전체 선택"이라 수식 하나로 모든 칸이 한 번에 계산됩니다.

NumPy 의 ndarray 는 Pandas, TensorFlow, PyTorch 가 모두 그 위에 지어진 **기반 데이터 구조**입니다. 오늘 이걸 배우면 이후 AI 과정 전체의 바닥재를 까는 셈입니다.

2. ndarray 와 4가지 속성

배열을 처음 만나면 항상 이 4가지부터 확인하는 습관을 들이세요.

(requests 에서 type => len => keys 를 확인했던 것과 같은 발상입니다.)

속성	뜻	예시 결과
<code>.ndim</code>	차원 수	2 (2차원)
<code>.shape</code>	각 차원의 크기	(2, 3) = 2행 3열
<code>.size</code>	전체 원소 개수	6
<code>.dtype</code>	원소의 데이터 타입	int64

```
arr = np.array([[1, 2, 3],
               [4, 5, 6]])

print(arr.ndim)      # 2
print(arr.shape)     # (2, 3)
print(arr.size)      # 6
print(arr.dtype)     # int64 (환경에 따라 int32 일 수 있음)
```

주의 - ndarray 는 내부의 모든 원소가 반드시 같은 데이터 타입이어야 합니다. np.array([1, 'a']) 처럼 섞어 넣으면 전부 문자열로 통일되어 버립니다. "숫자 계산이 안 되는데요?" 의 주된 원인입니다.

3. 배열을 만드는 함수들

값을 직접 나열하는 np.array() 외에, 틀을 빠르게 만드는 생성 함수들이 있습니다.

함수	동작	예시
<code>np.zeros(모양)</code>	모두 0	<code>np.zeros((2, 4))</code>
<code>np.ones(모양)</code>	모두 1	<code>np.ones((3, 3))</code>
<code>np.full(모양, 값)</code>	모두 지정한 값	<code>np.full((2, 3), 2)</code>
<code>np.arange(시작, 끝, 간격)</code>	연속 숫자 (파이썬 <code>range</code> 와 동일)	<code>np.arange(10, 30, 5) => [10 15 20 25]</code>
<code>np.linspace(시작, 끝, 개수)</code>	구간을 균일하게 n 등분	<code>np.linspace(0, 1, 5) => [0. 0.25 0.5 0.75 1.]</code>

```
print(np.zeros(3))          # [0. 0. 0.]
print(np.arange(5))         # [0 1 2 3 4]
print(np.linspace(0, 1, 5)) # [0. 0.25 0.5 0.75 1.]
```

`arange` 와 `linspace` 구분법: "간격"을 알면 `arange`, "개수"를 알면 `linspace`.

4. 인덱싱과 슬라이싱 - 그리고 View 라는 함정

1차원 배열은 파이썬 리스트와 사용법이 완전히 동일합니다. 새로 배울 것이 없습니다.

2차원 배열은 [행, 열] 형태로 접근합니다.

```
arr2d = np.array([[1, 2, 3, 4],
                  [5, 6, 7, 8],
                  [9, 10, 11, 12]])

print(arr2d[1, 2])          # 7          (1행 2열)
print(arr2d[1, :])          # [5 6 7 8] (1행 전체)
print(arr2d[:2, 1:3])       # [[2 3]
                             # [6 7]] (앞 2행 × 1~2열)
```

`arr2d[1][2]` 처럼 두 번 접근해도 되지만, 콤마를 쓴 `arr2d[1, 2]` 가 NumPy 표준이며 더 효율적입니다.

가장 중요한 함정: 슬라이싱은 복사가 아니라 **View** 다

- 파이썬 리스트: 슬라이싱하면 새로운 객체가 생깁니다. 잘라낸 것을 수정해도 원본은 무사합니다.
- NumPy 배열: 슬라이싱하면 원본의 일부를 들여다보는 창문(View)을 돌려줍니다. 창문 너머로 값을 바꾸면 원본이 바뀝니다.

```
python_list = [10, 20, 30, 40, 50]
sliced = python_list[1:4]
sliced[0] = 999
print(python_list)          # [10, 20, 30, 40, 50] => 원본 무사

numpy_array = np.array([10, 20, 30, 40, 50])
view = numpy_array[1:4]
view[0] = 999
print(numpy_array)          # [ 10 999 30 40 50] => 원본이 바뀜!
```

원본을 지키고 싶으면 `numpy_array[1:4].copy()` 로 명시적으로 복사하세요.

5. Boolean Indexing 과 Fancy Indexing

Boolean Indexing - 조건식으로 `True` 인 원소만 골라내는 방법. 오늘 이후 Pandas 필터링까지 이어지는 가장 중요한 문법입니다.

```
data = np.array([8000, 4500, 15000, 25000, 6000])

print(data > 10000)      # [False False  True  True False]  <= 조건의 정체는 bool 배열
print(data[data > 10000]) # [15000 25000]                  <= True 위치의 값만 추출

small = np.array([1, 2, 3, 4, 5, 6])
print(small[small % 2 == 0]) # [2 4 6]   짝수만
```

동작 원리 시각화 - 조건은 마스크(가림판)를 만들고, 그 마스크를 대괄호에 넣으면 구멍 뚫린 자리만 통과합니다.

```
data      : [ 8000  4500 15000 25000  6000 ]
data>10000: [ False False  True   True  False ]   <= 1단계: bool 마스크 생성
data[마스크]: [          15000 25000          ]   <= 2단계: True 만 통과
```

Fancy Indexing - 인덱스 목록을 넘겨 비연속적인 위치를 한 번에 뽑는 방법입니다.

```
arr = np.array([10, 20, 30, 40, 50, 60, 70])
print(arr[[0, 2, 5]])  # [10 30 60]
```

6. `reshape` - 모양 바꾸기

`arr.reshape(행, 열)` 은 원소의 총 개수를 유지한 채 배열의 구조만 바꿉니다.

```
arr = np.arange(12)      # 원소 12개

print(arr.reshape(3, 4))  # 3행 4열 (3 × 4 = 12, OK)
print(arr.reshape(4, -1)) # 4행, 열은 자동 계산 => (4, 3)
print(arr.reshape(-1, 2)) # 열 2개, 행은 자동 계산 => (6, 2)
```

- 변경 전후의 총 원소 개수가 반드시 같아야 합니다. `arr.reshape(5, 3)` 은 $12 \neq 15$ 라서 에러.
- `-1` 을 한 자리에 넣으면 남은 차원을 NumPy 가 대신 계산해 줍니다. ("나머지는 네가 알아서 채워")

7. 배열 연산과 브로드캐스팅

요소별 연산: 같은 위치의 원소끼리 계산됩니다. `for` 없이 한 줄로 끝나고 훨씬 빠릅니다.

```

단가 = np.array([4000, 4500, 5000])
수량 = np.array([2, 1, 3])

print(단가 * 0.9)      # [3600. 4050. 4500.] 배열 × 숫자: 모든 원소에 적용
print(단가 * 수량)     # [ 8000  4500 15000] 배열 × 배열: 같은 위치끼리

```

브로드캐스팅(Broadcasting): 모양이 달라도, 작은 쪽이 자동으로 확장되어 연산됩니다.

```

arr = np.array([[1, 2, 3],
                [4, 5, 6]])
print(arr + 5)
# [[ 6  7  8]
#   [ 9 10 11]]

```

여기서 숫자 5 는 내부적으로 `[[5, 5, 5], [5, 5, 5]]` 모양으로 **확장된 것처럼** 계산됩니다. 실제로 데이터를 복제하지는 않아서 메모리도 절약됩니다.

비유 - 도장 하나(작은 배열)로 서류 전체(큰 배열)에 찍는 것. 도장을 서류 수만큼 만들 필요가 없습니다.

이후 AI 과정에서 "편향(bias)을 더하는 연산" 이 바로 이 브로드캐스팅입니다. 지금은 "모양이 달라도 알아서 맞춰 계산된다" 까지만 기억하면 충분합니다.

8. 유니버설 함수 (ufunc) - 가볍게 훑기

배열의 모든 요소에 수학 함수를 한 번에 적용하는 함수들입니다. AI 과정에서 계속 만나므로 이름만 먼저 익혀 둡니다.

함수	동작
<code>np.sqrt(arr)</code>	각 요소의 제곱근
<code>np.exp(arr) / np.log(arr)</code>	지수 / 자연로그
<code>np.dot(A, B) (= A @ B)</code>	행렬 곱
<code>np.random.randn(행, 열)</code>	표준정규분포 난수 배열

```

print(np.sqrt(np.array([1, 4, 9, 16]))) # [1. 2. 3. 4.]

```

지금 단계에서는 `sqrt` 정도만 손에 익히고, 나머지는 "이런 게 있다" 로 넘어가도 됩니다.

9. 집계 함수와 axis

집계 함수: 배열 전체를 하나의 값으로 요약합니다.

```
매출 = np.array([8000, 4500, 15000, 25000, 6000])

print(매출.sum())      # 58500   합계
print(매출.mean())     # 11700.0 평균
print(매출.max())      # 25000   최댓값
print(매출.argmax())   # 3       최댓값의 "위치(인덱스)"
```

axis: 2차원 배열에서 "어느 방향으로 뭉갤 것인가" 를 지정합니다.

```
매출표 = [[8000, 4500, 15000],      <= 서울
          [6000, 5200, 10000]]     <= 부산
          (열0)  (열1)  (열2)
```

sum(axis=0): 세로 방향으로 눌러 합침 => 열마다 하나씩 => [14000 9700 25000] (메뉴별 합)
sum(axis=1): 가로 방향으로 눌러 합침 => 행마다 하나씩 => [27500 21200] (지역별 합)

헷갈릴 때의 암기법 - "axis 는 사라지는 방향". `axis=0` 이면 0번 축(행 방향)이 사라지고 열만 남습니다. 결과의 길이를 보면 역산할 수 있습니다: 결과가 열 개수만큼 나오면 `axis=0`.

Part B. Pandas

10. Pandas 란 - `Series` 와 `DataFrame`

Pandas 는 엑셀 스프레드시트처럼 생긴 데이터를 다루는 데 최적화된 파이썬 데이터 분석 라이브러리입니다.

Reason: 실무 데이터는 대부분 "행과 열이 있는 표" 입니다. NumPy 는 같은 타입의 숫자만 담지만, 표에는 문자열(지역명)과 숫자(단가)가 섞여 있습니다. Pandas 의 DataFrame 은 **컬럼마다 다른 자료형**을 허용해 이 문제를 해결합니다.

구조	차원	비유
<code>Series</code>	1차원	엑셀의 한 열(Column)
<code>DataFrame</code>	2차원	엑셀의 시트 전체 (Series 가 모이면 DataFrame)

엑셀 <=> Pandas 대응표

엑셀	Pandas
시트	DataFrame
열 하나	Series
필터	Boolean Indexing
피벗 테이블	<code>groupby</code> / <code>pivot_table</code>

11. DataFrame 만들기

가장 일반적인 방법은 딕셔너리입니다. 키가 열 이름, 값 리스트가 그 열의 데이터가 됩니다.

```
import pandas as pd

data = {
    '이름': ['Alice', 'Bob', 'Charlie'],
    '나이': [25, 30, 35],
    '점수': [85.5, 90.3, 78.9],
}
df = pd.DataFrame(data)
print(df)
#      이름  나이   점수
# 0   Alice   25  85.5
# 1    Bob   30  90.3
# 2 Charlie   35  78.9
```

왼쪽의 0, 1, 2 가 인덱스(index) - 행의 이름표입니다. 지정하지 않으면 0부터 자동으로 붙습니다.

기존 컬럼을 조합해 새 컬럼 만들기 - 실무에서 매일 쓰는 패턴입니다.

```
df['보정점수'] = df['점수'] + 5          # 열 전체에 한 번에 계산 (NumPy 벡터 연산 그대로!)
```

DataFrame 내부의 값은 사실 NumPy 배열입니다(df.values). 그래서 Part A 에서 배운 벡터 연산, boolean indexing 이 그대로 DataFrame 에서도 통합니다. 새 문법이 아니라 같은 문법의 재사용입니다.

12. Inspecting - 데이터와의 첫인사

처음 만난 데이터는 절대 추측하지 않고, 아래 순서로 확인합니다.

(requests 의 type => len => keys 습관의 Pandas 버전입니다.)

순서	메서드	알 수 있는 것
1	df.head(n) / df.tail(n)	위/아래 n행 미리보기 (기본 5행)
2	df.shape	(행 수, 열 수)
3	df.info()	컬럼별 타입, 결측치 여부, 메모리
4	df.describe()	숫자 컬럼의 통계 요약 (평균, 표준편차, 사분위수)
5	df['컬럼'].value_counts()	값별 빈도 (범주형 컬럼의 분포)

info() 를 읽는 법 - Non-Null Count 가 전체 행 수보다 작으면 그 컬럼에 결측치(빠진 값) 가 있다는 뜻입니다. 분석 전에 반드시 확인해야 하는 이유입니다.

describe() 의 사분위수 - 데이터를 크기순으로 세워 4등분한 지점입니다. 25% 값은 "하위 25% 경계", 50% 는 중앙값. 평균과 중앙값이 크게 다르면 한쪽으로 치우친 데이터라는 신호입니다.

13. 열 선택과 행 선택 - `[]`, `loc`, `iloc`

열(Column) 선택

```
df['이름']          # 한 열 => Series 반환
df[['이름', '점수']] # 여러 열 (리스트로!) => DataFrame 반환
```

대괄호가 두 겹인 이유: 바깥 `[]` 는 "선택한다", 안쪽 `[]` 는 "열 이름 리스트". `df['이름', '점수']` 로 쓰면 에러입니다.

행(Row) 선택 - `loc` 과 `iloc` 의 구분이 핵심입니다.

	<code>loc</code>	<code>iloc</code>
기준	이름(label)	위치(정수)
슬라이싱 끝점	포함	미포함 (파이썬과 동일)
예시	<code>df.loc['a':'c']</code> => a, b, c	<code>df.iloc[0:3]</code> => 0, 1, 2행

```
df.loc[df['지역'] == '서울', '매출'] # 행 조건 + 열 이름을 동시에
df.iloc[0, 1]                        # 0행 1열 (순수 위치)
```

암기법 - `iloc` 의 **i** 는 **integer**. 숫자로 찍으면 `iloc`, 이름으로 부르면 `loc`.

`loc` 슬라이싱은 파이썬 상식과 다르게 끝점을 포함합니다. `df.loc['a':'c']` 에 'c' 행이 들어옵니다. 시험(과 실무)에 정말 잘 나오는 함정입니다.

14. 조건 필터링

Part A 5장의 Boolean Indexing 이 DataFrame 으로 확장된 것입니다. 원리가 완전히 같습니다.

```
df[df['점수'] >= 90] # 조건이 True 인 "행"만 남김
```

복합 조건 - 규칙 두 가지만 지키면 됩니다.

```
# 1) and/or 가 아니라 & / | 를 쓴다
# 2) 각 조건을 반드시 괄호로 감싼다
df[(df['점수'] >= 80) & (df['과목'] == '수학')] # 그리고
df[(df['점수'] == 92) | (df['과목'] == '과학')] # 또는
df[~(df['과목'] == '과학')]                  # 부정(NOT)
```

괄호를 빼면 연산자 우선순위 때문에 `&` 가 먼저 계산되어 `TypeError` 가 납니다. "복합 조건 = 괄호 + `&` / `|`" 를 한 몸으로 외우세요.
`and/or` 는 Pandas 에서 동작하지 않습니다.

특수 조건 필터 - 자주 쓰는 3종 세트:

메서드	용도	예시
<code>.isin(리스트)</code>	값이 목록에 포함되는 행	<code>df[df['학년'].isin([2, 3])]</code>
<code>.str.contains('글자')</code>	특정 글자를 포함하는 행	<code>df[df['메뉴'].str.contains('라떼')]</code>
<code>.isnull()</code> / <code>.notnull()</code>	결측치인 / 아닌 행	<code>df[df['점수'].notnull()]</code>

`isnull()` 과 `notnull()` 은 **인자 없이** 호출하는 메서드입니다. `df['점수'].isnull()` 처럼 씁니다.

15. 전처리 - 결측치와 타입 다루기

실무 데이터에는 항상 구멍(결측치, `NaN`)이 있습니다. 두 가지 선택지가 있습니다.

메서드	동작	트레이드오프
<code>df.dropna()</code>	결측치가 있는 행(또는 <code>axis=1</code> 로 열)을 제거	확실하지만 데이터 손실
<code>df.fillna(값)</code>	결측치를 특정 값으로 대체	손실이 적어 일반적으로 선호

타입 변환 - 숫자가 문자열로 읽혀 온 경우(CSV 에서 혼함) `astype` 으로 바로잡습니다.

```
df['점수'] = df['점수'].fillna(0)           # 1) 결측치를 먼저 채우고
df['점수'] = df['점수'].astype(int)        # 2) 정수형으로 변환
```

순서 주의 - `NaN` 이 남아 있는 채로 `astype(int)` 를 부르면 에러가 납니다. **fillna 먼저, astype 나중**. 실습 노트북에서도 이 순서 그대로 나옵니다.

16. `groupby` 와 `pivot_table` - 표를 요약

오늘 실습 노트북(Step 7)에 등장하는 내용입니다. 여기까지 오면 실습의 마지막입니다.

groupby: "같은 값끼리 묶어서 요약하라". 엑셀 피벗 테이블, SQL 의 `GROUP BY` 와 같은 발상입니다.

```
df.groupby('지역')['매출'].sum()           # 지역별 총매출
df.groupby('메뉴')['단가'].mean()         # 메뉴별 평균 단가
```

동작 원리 3단계 (Split - Apply - Combine)

원본 표	1) Split: 지역별로 쪼갬다	2) Apply: 각각 합한다	3) Combine: 다시 합친다
지역 매출	서울: [8000, 15000]	서울: 23000	지역
서울 8000 =>	부산: [6000, 10000]	=> 부산: 16000 =>	서울 23000
부산 6000			부산 16000
서울 15000			
부산 10000			

여러 집계를 한 번에 - `agg` 에 새이름=('컬럼' , '함수') 형태로 나열합니다.

```
df.groupby('지역').agg(
    매출합=( '매출' , 'sum' ),
    매출평균=( '매출' , 'mean' ),
)
```

pivot_table: `groupby` 와 같은 일을 하되, 한 기준을 가로(columns)로 펼쳐 교차표를 만듭니다.

```
df.pivot_table(index='지역', columns='메뉴', values='매출', aggfunc='sum')
# 메뉴      바닐라라떼  아메리카노  카페라떼
# 지역
# 광주      ...          ...          ...
# 서울      ...          ...          ...
```

구분법 - 결과를 세로로 길게 보고 싶으면 `groupby`, 바둑판(교차표)으로 보고 싶으면 `pivot_table`.

정리

1. NumPy 배열의 연산은 **원소별로 한 번에** 적용된다. 반복문은 잊어라. (벡터화)
2. NumPy 슬라이싱은 복사가 아니라 **View** 다. 원본을 지키려면 `.copy()`.
3. `axis=0` 은 세로로 눌러 **열별** 결과, `axis=1` 은 가로로 눌러 **행별** 결과.
4. Pandas 필터링은 `df[(조건1) & (조건2)]` - **괄호 + `&` / `|`**, `and/or` 금지.
5. `loc` 은 이름 기반(끝점 포함!), `iloc` 은 위치 기반(끝점 미포함). **i 는 integer**.

다음 단계 - `numpy_pandas_guide.py` 를 CHAPTER 순서대로 실행하며 출력을 확인한 뒤, 실습 노트북의 `TODO 1` 부터 도전하세요. 막히면 이 문서의 해당 장 번호로 돌아오면 됩니다.